

AI Agent 应用开发学习路线图 (2025-2026)

整合自 GitHub 开源仓库、B站/YouTube 教程、官方文档、技术博客的完整学习路线。 最后更新：2026-06-03

第一阶段：基础预备（Stage 0，2-4 周）

在进入 Agent 开发之前必须打好的基础：

技能	要求	学习资源
Python	函数、类、异常处理、async/await、API 调用	廖雪峰 Python 教程 / Python 官方文档
LLM 基础	Token、Context Window、Temperature、API 调用模式	OpenAI/Anthropic 官方文档
向量与嵌入	Embedding 原理、向量相似度搜索	Pinecone/Chroma 文档
命令行	Shell 基本操作、环境变量	任何 Linux 入门教程
Git	clone/commit/push/branch	Git 官方教程

⚠ 不需要：从头训练模型、深入理解反向传播、Transformer 内部实现

第二阶段：LLM 与 Prompt Engineering（Stage 1-2，2-4 周）

Stage 1：LLM API 基础（1 周）

- 调用 OpenAI / Anthropic / 国产模型 API 完成对话
- 让模型输出结构化 JSON（Function Calling）
- 理解 Token 计费与 API 使用模式

Stage 2：Prompt Engineering（1-2 周）

- **System Prompt** 设计与调试
- **Few-Shot Prompting**：给模型几个示例让它学会格式
- **Chain-of-Thought (CoT)**：让模型展示推理过程
- **Context Engineering**：如何管理上下文窗口

 视频推荐：

- B站：[吴恩达《ChatGPT Prompt Engineering for Developers》中文版](#)
- B站：[1小时彻底吃透Agent智能体 — 快速入门](#)

✂ 第三阶段：Tool Use 与第一个 Agent (Stage 3 , 2-3 周)

☆

这是最关键的阶段——理解 Agent 的核心循环。

核心概念：ReAct 模式

Think → Act → Observe → Think → Act → Observe → ... → Final Answer

手写一个最小 Agent (50-150 行)

核心循环伪代码

```
def agent_loop(user_query, tools, max_steps=10):
    messages = [{"role": "user", "content": user_query}]
    for step in range(max_steps):
        response = llm.chat(messages, tools=tools)
        if response.has_tool_call():
            tool_result = execute_tool(response.tool_call)
            messages.append({"role": "tool", "content": tool_result})
        else:
            return response.content # 最终答案
    return "Max steps reached"
```

关键产出

- 一个能调用工具、执行动作、返回答案的最小 Agent
- 加上最大步数限制、超时处理和错误恢复

📺 视频推荐：

- YouTube：[Anthropic - Building effective agents](#)
- B站：[保姆级AI Agent智能应用从0到1定制开发](#) — 含 MCP 协议讲解

✂ 第四阶段：Agent 框架 (Stage 4 , 2-3 周)

2026 年主流框架对比

框架	定位	核心优势	适用场景	GitHub Stars
LangGraph	生产级状态机框架	有向图模型、状态管理、检查点恢复、可审计	复杂条件分支、长运行工作流	高
CrewAI	多 Agent 角色协作	角色分配、少写 40-60% 代码、快速原型	多角色协作、内容创作流水线	高
Anthropic Agent SDK	Claude 原生 Agent SDK	最深 MCP 集成、原生文件/Shell 访问、Hooks 系统、Skills 市场	编码 Agent、代码审查、仓库自动化	~6.6k
OpenAI Agents SDK	轻量级多 Agent SDK	最干净的 Handoff 模型、100+ LLM 支持、内置语音、沙箱执行	快速原型、多 Agent 编排、语音 Agent	~25.7k

Microsoft Agent Framework	企业级 Agent 框架	Azure 生态集成、AutoGen + Semantic Kernel 合并	Azure 企业场景	中
Google ADK	Google 生态 Agent SDK	Gemini 原生、Google Cloud 集成	GCP 生态	中

低代码/无代码平台对比

平台	定位	核心优势	最适合
Dify	开源企业级 AI 应用平台	AI 原生、RAG 最强、20+ 模型支持、私有化部署	知识库问答、智能客服、RAG 应用
Coze (扣子)	字节跳动零代码平台	零代码最易用、抖音/飞书/微信多端发布	快速原型、运营人员、非技术用户
n8n	开源工作流自动化工具	400+ 集成节点、完全开源、最灵活	复杂系统集成、技术团队化

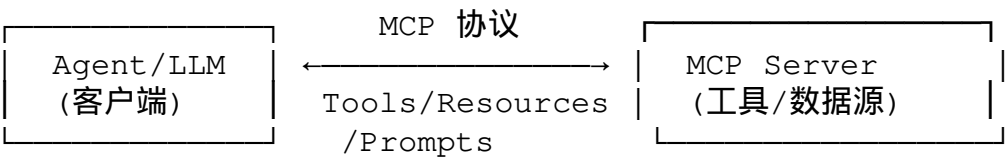
📺 视频推荐：

- B站：[翻遍整个B站，这绝对是2025讲的最好的AI Agent智能体教程](#) — 含 LangGraph Memory
- B站：[2025B站最新版Agent智能体开发教程，18个练手项目](#)

🔑 第五阶段：MCP 协议与现代 Agent 生态 (Stage 5 , 3-4 周) ☆☆

MCP (Model Context Protocol) — 2026 年最重要的 Agent 协议

MCP 已被捐赠给 Linux 基金会，OpenAI、Google、Microsoft、AWS、Salesforce 均表示支持。



MCP 三种核心能力

1. **Tools**：可执行函数（如搜索、计算、读文件）
2. **Resources**：暴露数据（如数据库内容、API 响应）
3. **Prompts**：可复用的 LLM 提示模板

快速开始：用 Python FastMCP 写一个 MCP Server

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("MyAgent")

@mcp.tool()
def search_knowledge_base(query: str) -> str:
    """Search the internal knowledge base"""
```

```
# 你的搜索逻辑
return f"Results for: {query}"

@mcp.resource("doc://{doc_id}")
def get_document(doc_id: str) -> str:
    """Get a document by ID"""
    return f"Document {doc_id} content..."

mcp.run(transport='stdio')
```

当前生态

- 200+ 预构建的 MCP Server (文件系统、数据库、API、浏览器等)
- MCP Inspector 用于测试和调试
- Claude Desktop / Cursor / VS Code 原生支持

📺 视频推荐：

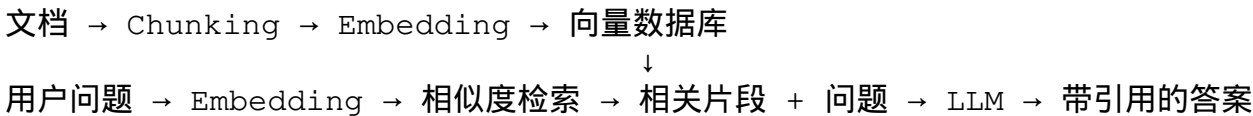
- LearnAI Singapore：[Build Your First MCP Server in 6 Steps](#)
- B站：[保姆级AI Agent教程](#) — 含高德 MCP、Playwright MCP、GitHub MCP 实战

🔑 第六阶段：RAG 与记忆系统 (Stage 6 , 2 周)

Agent 的三种记忆

记忆系统架构	
短期记忆	Context Window (对话历史)
长期记忆	向量数据库 (跨会话信息)
情景记忆	特定事件/经验的存储和检索

RAG workflow



常用向量数据库

- FAISS：Meta 开源，本地运行 (入门首选)
- Chroma：轻量级，Python 友好
- Pinecone：托管服务，生产级
- Weaviate：开源，混合搜索
- Qdrant：Rust 编写，高性能

关键产出

一个资料研究助手：输入主题 → 自动搜索 → 筛选 → 总结 → 输出引用链接

✈ 第七阶段：多 Agent 与生产化（Stage 7，2-4 周）

多 Agent 设计模式

模式	描述	适用场景
Planner → Executor	一个规划，多个执行	复杂任务分解
Supervisor 模式	一个主管分配任务给子 Agent	多步骤工作流
Graph 模式	有向图定义 Agent 间流转	LangGraph 典型场景
Handoff 模式	Agent 间声明式交接	OpenAI Agents SDK
Critic/Reviewer	一个生成，一个审核	内容创作、代码审查

生产化要点

- 评估体系：成功率、延迟、成本、Trace 分析（LangSmith、OpenAI Evals）
- 安全：危险操作人工确认、权限边界、Prompt Injection 防护
- 可观测性：OpenTelemetry、Prometheus + Grafana
- 预算控制：Token 消耗上限、步数限制

📺 视频推荐：

- YouTube：[Anthropic - Equipping agents for the real world with Agent Skills](#)

✈ 第八阶段：Computer Use & Browser Use（Stage 8，2-3 周）

- **Browser Agent**：Playwright / browser-use 自动化网页操作
- **Computer Use**：屏幕截图 → 分析 → 操作桌面应用
- **Code Sandbox**：安全执行环境（Docker / E2B）
- 安全限制：操作边界、确认机制、失败恢复

🎯 项目阶梯（从易到难，10 个实战项目）

级别	项目	学习目标	预估时间
1	Calculator Agent	最小 Tool Call Loop	1 天
2	Web Research Agent	搜索→筛选→总结→引用	2-3 天
3	PDF QA Agent	RAG、Chunking、Retrieval	3-5 天
4	Code Review Agent	读 diff、风险分析、建议	3-5 天
5	Browser Agent	页面自动化、点击、提取	5-7 天
6	Mini Coding Agent	Shell、文件编辑、权限、Session	1-2 周
7	Multi-Agent Writer	Planner→Writer→Reviewer 协作	1-2 周
8	MCP Server 发布	写一个可复用 MCP Server	1 周
9	Personal Knowledge Agent	长期记忆、多入口、Skills	2-3 周

📺 视频教程推荐汇总

B站（中文）

教程	链接	特色
翻遍整个B站最好的Agent教程	BV1TKWRz3ETe	企业级 + LangGraph Memory + LangMem SDK
18个Agent练手项目	BV1yXahzFEBg	LangChain + RAG + FastAPI 部署
保姆级MCP协议实战	BV12Fn3zyEC9	MCP + Cursor + Redis/MongoDB
Agent企业级项目搭建	BV1mKKVz7EjQ	ReAct + RAG + 阿里云百炼
1小时彻底吃透Agent	BV18BWxzrELV	快速入门 + Coze 实操
LLM/Agent/RAG全栈	BV145TvzLEp6	含 Dify、提示词工程

YouTube（英文）

频道/教程	内容	难度
Anthropic 官方	Building Effective Agents、Agent Skills	中
LangChain 官方	LangGraph 教程系列	中-高
freeCodeCamp	LangGraph Full Course	入门-中
Sam Witteveen	Ollama + Agent 实战	入门-中
Cole Medin	AI Agent 全栈教程	中

📁 GitHub 开源仓库推荐

仓库	说明	Stars
Datawhale Agent-Learning-Hub	AI Agent 学习路线与资料库，9 阶段学习	☆
awesome-agentic-ai-zh	8 阶段 145+ 精选项目的结构化路线图	☆
agent-study	36 章节、22000+ 行代码的深度 Agent 教程	☆
agent-first-dev	8 周自学路径，"Learn by Doing"	☆
awesome-llm-apps	LLM 应用合集，含多个 Agent 项目	☆
GPT Researcher	自主研究 Agent 参考实现	☆

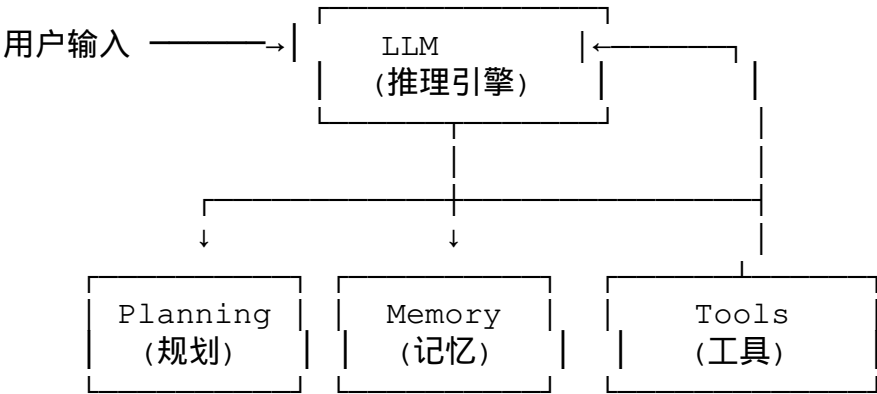
📖 必读文章 & 官方文档

资源	链接	重要性
Anthropic: Building Effective Agents	anthropic.com/engineering	☆☆☆☆☆
OpenAI: A Practical Guide to Building Agents	platform.openai.com	☆☆☆☆☆
LangGraph 官方文档	langchain-ai.github.io/langgraph	☆☆☆☆
MCP 官方规范	modelcontextprotocol.io	☆☆☆☆

Anthropic Agent SDK 文档	platform.claude.com	☆☆☆
CrewAI 官方文档	docs.crewai.com	☆☆☆
Dify 官方文档	docs.dify.ai	☆☆☆

📁 核心概念速查

Agent = LLM + Planning + Memory + Tools



两大主流 Agent 架构

架构	描述	优点	缺点
ReAct	Thought → Action → Observation 循环	灵活、自适应	可能循环、Token 消耗大
Plan-and-Solve	先生成完整计划，再逐步执行	可预测、可审计	对意外情况适应差

关键设计原则

1. 先构建小的、可靠的 **Agent**，比华而不实的 Demo 有价值
2. 使用严格 **Schema** 的工具——模糊的工具定义是第一大失败原因
3. 加评测再加 **Agent**——没有 Eval、Trace、权限的 Agent 只是 Demo
4. 高风险操作留人工在环——发邮件、删文件、付款需要确认
5. 如果普通脚本能解决，就不要用 **Agent**——Agent 适合不确定性高的任务

🌐 推荐学习路径

路径 A：CLI Power User（8-10 周）

适合：想用 Agent 工具提升效率，不打算从零写 Agent

1. Stage 0-2：基础 + Prompt Engineering（3-5 周）
2. 选择 Claude Code / Cursor / Codex CLI 开始用（1 周）
3. 配置 CLAUDE.md、Slash Command、多步骤任务（1-2 周）
4. MCP 接入、CI/CD 自动化（1-2 周）

路径 B：Agent Builder（5-7 个月）

适合：想从零构建自己的 Agent 系统

1. Stage 0-2：基础（3-5 周）
2. Stage 3：Tool Use + Hello Agent（2-3 周）
3. Stage 4：挑一个框架深入（2-3 周）
4. Stage 5：MCP 协议 + Agent 生态（3-4 周）
5. Stage 6：RAG + 记忆系统（2 周）
6. Stage 7：多 Agent + 生产化（2-4 周）
7. Stage 8：Computer Use（2-3 周）

路径 C：低代码入门（2-4 周）

适合：非技术背景，想快速搭建 AI 应用

1. Week 1：用 Coze 搭建第一个聊天机器人
2. Week 2：用 Dify 搭建知识库问答 + RAG
3. Week 3：学习 n8n 串联多系统自动化
4. Week 4：基于以上平台做一个小型项目

✂ 学习方法论

1. **Learn by Doing** — 不要陷入“只看不写”的陷阱，30% 看 + 70% 写
2. **Just-in-Time Learning** — 需要什么学什么，不要试图先把所有东西学完
3. 每完成一个项目就打勾 — 用项目驱动，而非课程驱动
4. 追踪每一次运行 — 知道失败发生在哪个环节
5. 从 Claude Code 学架构 — 它是当前最好的 Agent 工程样本

📋 阶段性 Checklist

- ☐ 基础：Python async/await、LLM API、Embedding、Git
- ☐ **Prompt**：System Prompt、Few-Shot、CoT
- ☐ **Hello Agent**：手写 ReAct 循环（50-150 行）
- ☐ 框架：选一个框架做项目（推荐先 LangGraph 或 Anthropic SDK）
- ☐ **MCP**：写一个 MCP Server，在 Claude Desktop/Cursor 中使用
- ☐ **RAG**：搭建一个带引用的知识问答系统
- ☐ **Memory**：实现跨会话的长期记忆
- ☐ 多 **Agent**：Planner + Executor + Reviewer 协作
- ☐ 生产化：Evals + Trace + 权限 + 成本控制
- ☐ 发布：将 Agent 部署为 API 或 MCP Server

🔔 一句话建议：从 [Datawhale Agent-Learning-Hub](#) 开始，按 Stage 0→8 打勾推进；同时强力推荐投入 Claude Code 生态——它是 2026 年最值得深入研究的 Agent 工程样本。